

GriidAi

Export Conversion Engine

Frontend Implementation Plan

Technical Specification — v1.0

Document owner GriidAi Frontend Team	Status Ready for Implementation
Date March 2026	Estimated effort 5–7 working days

1. Overview & Objectives

This document details the frontend implementation plan for the GriidAi Export Conversion Engine. It covers every UI change required: removing existing placeholder disclaimers, adding the Export As button, building the format selection modal, implementing async polling, and surfacing the success notification with a download link.

GOAL	Transition the file management UI from a static 'Coming Soon' state to a fully functional export experience — with format selection, async progress feedback, and a one-click download when conversion completes.
-------------	---

1.1 Scope

Dimension	Details
In scope	Remove 'Coming Soon' banners, add Export As button, build format selection modal, polling logic, success/error toast notifications, format compatibility enforcement in UI
Out of scope	Backend conversion workers, API endpoint implementation, authentication, billing metering, file upload or preview features
Primary stack	React 18 · TypeScript · TailwindCSS · React Query (polling) · Headless UI or Radix (modal)
Depends on	Backend /export and /export/status/{task_id} endpoints being live (can mock during development)

Dimension	Details
Estimated effort	5–7 frontend engineering days

1.2 Affected UI Areas

UI Area	Change Required
File list / table row	Add 'Export As...' button to each row action menu
File detail / preview panel	Add 'Export As...' button to the header action bar
Download alert / modal (existing)	Remove 'Coming Soon' disclaimer text and badge
New: Export modal	Build from scratch — format selector with vector/raster grouping
New: Export progress indicator	Inline progress state in the modal while conversion runs
New: Success toast notification	Shows 'Export ready' with a timed download link (24h expiry)
New: Error toast notification	Shows readable failure reason with a retry option

2. Component Architecture

All new export-related UI is encapsulated in a self-contained feature folder. This keeps the export feature isolated from the rest of the codebase and makes it easy to test independently.

2.1 Folder Structure

```
src/
  features/
    export/
      components/
        ExportButton.tsx           // trigger button (reusable)
        ExportModal.tsx           // modal shell + state machine
        FormatSelector.tsx        // radio group – vector/raster sections
        ExportProgress.tsx        // progress bar + status message
        ExportSuccessToast.tsx    // success notification with download link
        ExportErrorToast.tsx     // failure notification with retry
      hooks/
        useExport.ts              // initiates export, returns task_id
        useExportStatus.ts        // polling hook – wraps React Query
      types/
        export.types.ts           // ExportFormat, ExportStatus, FileType enums
      constants/
        formats.ts                // format definitions, icons, compatibility
  map
  api/
    exportApi.ts                 // POST /export and GET /export/status
  wrappers
```

2.2 State Machine

The export modal has five distinct states. Transitions are one-directional except for the reset path (close or retry):

State	Description / User sees
IDLE	Modal not open. ExportButton visible in file row.
SELECTING	Modal open. User sees format options. Export button enabled.
QUEUED	Export button clicked. API returned 202. Spinner + 'Your export is being prepared...'
CONVERTING	Worker picked up task. Progress bar advancing 0 → 100. Status message from API.
SUCCESS	Task complete. Modal closes. Success toast appears with download link.
FAILED	Task failed. Error message shown inside modal. Retry button visible.

```
type ExportState =  
  | { status: 'idle' }  
  | { status: 'selecting' }  
  | { status: 'queued'; taskId: string }  
  | { status: 'converting'; taskId: string; progress: number; message: string }  
  | { status: 'success'; downloadUrl: string; expiresAt: string }  
  | { status: 'failed'; error: string }
```

3. Remove 'Coming Soon' Disclaimers

FIRST TASK

This is a quick win — ship it independently before the modal work so the UI stops misleading users the moment the backend is ready.

3.1 Find All Instances

Search the codebase for every occurrence of the Coming Soon pattern before touching any code:

```
// Terminal  
grep -r "Coming Soon\|coming-soon\|comingSoon" src/ --include="*.tsx" --  
include="*.ts"
```

3.2 Changes per Location

File / Component	Change
DownloadAlert.tsx	Remove <Badge variant='warning'>Coming Soon</Badge> and the disclaimer paragraph beneath it
FileActionMenu.tsx	Remove disabled prop and tooltip 'Export coming soon' from the Download menu item
FileDetailHeader.tsx	Remove the grey 'Export formats coming soon' helper text under the download button
Any CSS / Tailwind class	Remove opacity-50 or cursor-not-allowed classes applied to download-related buttons

NOTE

Do not delete the existing download button components — they will be reused as the trigger for the new Export As modal. Only remove the placeholder copy and disabled states.

4. ExportButton Component

A small, reusable button that opens the export modal. It is placed wherever the old Coming Soon download button lived — no new positions, no layout changes.

4.1 Props Interface

```
interface ExportButtonProps {
  fileId: string
  fileType: 'vector' | 'raster' // drives format availability in modal
  variant?: 'primary' | 'ghost' // primary in detail header, ghost in table
  row
  size?: 'sm' | 'md'
}
```

4.2 Behaviour

- Clicking the button sets modal state to SELECTING and opens ExportModal
- Button is never disabled (Coming Soon state removed). If the file is still processing/uploading, show a tooltip: 'File is still processing — try again shortly'
- Only one export modal can be open at a time — if a modal is already open for a different file, close it first

4.3 Code Pattern

```
export function ExportButton({ fileId, fileType, variant = 'ghost', size = 'sm'
}: ExportButtonProps) {
  const { openExportModal } = useExportModalStore()
  return (
    <button
      onClick={() => openExportModal(fileId, fileType)}
      className={clsx(buttonVariants({ variant, size })), "gap-1.5")}
    >
      <ArrowDownTrayIcon className='h-4 w-4' />
      Export As...
    </button>
  )
}
```

5. FormatSelector Component

The core of the modal — a grouped radio selection showing available formats based on the file type. Incompatible formats are shown as disabled with a clear reason, not hidden.

5.1 Format Definitions (formats.ts)

```
export const EXPORT_FORMATS = [
  {
    id: 'geojson',
    label: 'GeoJSON',
    extension: '.geojson',
    description: 'Web-friendly, human-readable',
    group: 'vector',
  },
  { id: 'shapefile', label: 'ESRI Shapefile', extension: '.shp',
    description: 'ArcGIS & QGIS compatible', group: 'vector' },
  { id: 'kml', label: 'KML', extension: '.kml',
    description: 'Google Earth & Maps', group: 'vector' },
  { id: 'geotiff', label: 'GeoTIFF', extension: '.tif',
    description: 'Full metadata preserved', group: 'raster' },
  { id: 'png', label: 'PNG + World File', extension: '.png',
    description: 'With .pgw georeferencing', group: 'raster' },
  { id: 'jpeg', label: 'JPEG', extension: '.jpg',
    description: 'Compressed, no alpha', group: 'raster' },
] as const
```

5.2 Compatibility Logic

The FormatSelector receives fileType and disables incompatible groups entirely. This prevents API-level 400 errors from ever reaching the user:

```
function isFormatAvailable(format: ExportFormat, fileType: FileType): boolean {
  if (fileType === 'vector') return format.group === 'vector'
```

```
    if (fileType === 'raster') return format.group === 'raster'
    return false
  }
```

5.3 Visual Design Rules

- Available formats: white card, full opacity, radio input selectable
- Unavailable formats: grey card, 40% opacity, radio input disabled, tooltip on hover: 'Not available for [vector/raster] files'
- Selected format: purple border (1.5px solid #534AB7), subtle purple background tint
- Group labels: all-caps, small, muted — 'VECTOR FORMATS' / 'RASTER FORMATS'
- Default selected format: first available format for the given file type

6. ExportModal Component

6.1 Modal Shell

Built on Radix UI Dialog or Headless UI for accessibility (focus trap, ESC to close, aria labels). The modal content changes based on the current ExportState.

Modal section	Content
Header	Title: 'Export As...' — subtitle: filename of the file being exported
Body (SELECTING)	FormatSelector component — grouped radio cards
Body (QUEUED / CONVERTING)	ExportProgress component — spinner or progress bar + status message
Body (FAILED)	Error message paragraph + Retry button + Close button
Footer (SELECTING)	Cancel button (ghost) + Export button (primary, disabled until format selected)
Footer (QUEUED / CONVERTING)	Cancel button only — Export button replaced by progress indicator
Footer (SUCCESS / FAILED)	No footer — modal closes on success, failure handled inline

6.2 Modal Dimensions & Behaviour

- Width: 480px on desktop, 100vw on mobile (with 16px side padding)
- Height: auto, min 320px
- Backdrop: semi-transparent black overlay (rgba 0,0,0,0.45)
- Closing while QUEUED or CONVERTING: show confirmation dialog — 'Your export will continue in the background. You can close this and we will notify you when it is ready.'
- Closing while SELECTING: close immediately, no confirmation

6.3 Export Button click handler

```

async function handleExport() {
  setState({ status: 'queued', taskId: '' })
  try {
    const { taskId } = await exportApi.initiateExport(fileId, selectedFormat)
    setState({ status: 'queued', taskId })
    // polling starts automatically via useExportStatus hook
  } catch (err) {
    setState({ status: 'failed', error: getReadableError(err) })
  }
}

```

7. Polling — useExportStatus Hook

Polls GET /export/status/{task_id} until the task reaches a terminal state (SUCCESS or FAILED). Built on React Query's refetchInterval for clean lifecycle management.

7.1 Hook Implementation

```

import { useQuery } from '@tanstack/react-query'

export function useExportStatus(taskId: string | null) {
  return useQuery({
    queryKey: ['export-status', taskId],
    queryFn: () => exportApi.getStatus(taskId!),
    enabled: !!taskId,
    refetchInterval: (data) => {
      if (!data) return 3000 // poll every 3s
      if (['success', 'failure'].includes(data.status)) return false // stop
      return 3000
    },
    retry: 3, // retry on network error
  })
}

```

7.2 Timeout Handling

If the task has been in QUEUED or CONVERTING state for more than 10 minutes without reaching a terminal state, the hook stops polling and surfaces a timeout error to the user:

```

const POLL_TIMEOUT_MS = 10 * 60 * 1000 // 10 minutes

// Inside the hook, track startTime
refetchInterval: (data) => {
  if (Date.now() - startTime > POLL_TIMEOUT_MS) {
    setTimeout() // sets state to FAILED with timeout message
  }
}

```

```

    return false
  }
  ...
}

```

7.3 Status → State Mapping

API status value	Modal state transition
'queued'	Keep state as QUEUED. Show spinner. No progress value yet.
'progress' + progress %	Transition to CONVERTING. Update progress bar and message.
'success' + download_url	Transition to SUCCESS. Close modal. Fire success toast.
'failure' + error	Transition to FAILED. Keep modal open. Show error inline.
Network error (fetch failed)	Do not change state. React Query retries up to 3 times before surfacing.

8. ExportProgress Component

Displayed inside the modal body during QUEUED and CONVERTING states.

8.1 Visual Design

- QUEUED state: animated spinner (Tailwind animate-spin) centred in the modal body. Text: 'Your export is being prepared...'
- CONVERTING state: horizontal progress bar (purple fill, grey track, rounded). Percentage text right-aligned. Status message from API below bar — e.g. 'Converting to GeoJSON...' or 'Writing Shapefile...'
- Progress bar animates smoothly — use CSS transition on width (duration 400ms ease-out)
- Do not show a numeric percentage during QUEUED — the bar appears only once progress > 0 is received from the API

8.2 Code Pattern

```

export function ExportProgress({ status, progress, message }:
ExportProgressProps) {
  if (status === 'queued') return (
    <div className='flex flex-col items-center gap-3 py-8'>
      <Spinner className='h-8 w-8 text-purple-600' />
      <p className='text-sm text-muted-foreground'>Your export is being
prepared...</p>
    </div>
  )
}

```



```
    return (
      <div className='space-y-2 py-4'>
        <div className='flex justify-between text-sm'>
          <span className='text-muted-foreground'>{message}</span>
          <span className='font-medium'>{progress}%</span>
        </div>
        <div className='h-2 w-full rounded-full bg-gray-100'>
          <div
            className='h-2 rounded-full bg-purple-600 transition-all duration-
400'
            style={{ width: `${progress}%` }}
          />
        </div>
      </div>
    )
  }
}
```

9. Toast Notifications

9.1 Success Toast

Fires when the polling hook receives status = 'success'. The modal closes first, then the toast appears. The toast must persist until the user dismisses it or downloads the file — do not auto-dismiss because the user needs to click the link.

Property	Value
Title	Export ready
Body	'[filename].[ext] is ready to download. Link expires in 24 hours.'
CTA button	'Download' — opens signed URL in a new tab
Duration	Persistent — no auto-dismiss. User must click Download or X.
Position	Bottom-right (or top-right — match existing app toast position)
Icon	Green checkmark circle

9.2 Error Toast

Fires if the task reaches FAILED state or if the 10-minute timeout is hit.

Property	Value
Title	Export failed
Body	The readable error message returned by the API. If API gave a technical error, show: 'Something went wrong. Please try again.'

Property	Value
CTA button	'Try Again' — re-opens the Export modal at SELECTING state
Duration	Persistent — no auto-dismiss
Icon	Red X circle

9.3 Background Export Notification

If the user closes the modal while a conversion is in progress, polling continues in the background using a global export store (Zustand or React context). When the task completes, the success or error toast fires regardless of what page the user is on.

```
// Global export store keeps track of all in-flight tasks
interface ExportStore {
  tasks: Map<string, ExportTask>
  addTask: (taskId: string, filename: string, format: string) => void
  resolveTask:(taskId: string, result: ExportResult) => void
}
```

10. API Layer (exportApi.ts)

Thin wrappers around fetch — no business logic. All error handling happens in the hooks and components.

10.1 initiateExport

```
export async function initiateExport(
  fileId: string,
  format: ExportFormatId
): Promise<{ taskId: string }> {
  const res = await fetch('/api/export', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ file_id: fileId, target_format: format }),
  })
  if (!res.ok) {
    const err = await res.json().catch(() => ({}))
    throw new ExportApiError(res.status, err.detail ?? 'Export request failed')
  }
  const data = await res.json()
  return { taskId: data.task_id }
}
```

10.2 getStatus

```
export async function getStatus(taskId: string): Promise<ExportStatusResponse>
{
```

```

const res = await fetch(`/api/export/status/${taskId}`)
if (!res.ok) throw new ExportApiError(res.status, 'Status check failed')
return res.json()
}

```

10.3 Error Mapping

HTTP status	User-facing message
400	'This format is not compatible with the selected file.'
404	'File not found. Please refresh and try again.'
413	'This file is too large to export. Please contact support.'
429	'Too many exports in progress. Please wait a moment and try again.'
500 / 502 / 503	'Our export service is temporarily unavailable. Please try again shortly.'
Network error	'Could not reach the server. Please check your connection.'

11. Accessibility Requirements

REQUIRE D

All export UI must meet WCAG 2.1 AA. The modal in particular must be fully keyboard-operable and screen-reader-friendly.

Requirement	Implementation
Focus trap	When modal opens, focus moves to first interactive element (first radio option). Tab cycles within modal only.
ESC to close	Closes modal if in SELECTING state. Shows confirmation if QUEUED/CONVERTING.
ARIA role	Dialog element has role='dialog' aria-modal='true' aria-labelledby pointing to modal title.
Radio group	FormatSelector uses role='radiogroup' with aria-label='Select export format'. Each option has aria-label='[Format] — [description]'.
Disabled formats	Disabled radio inputs have aria-disabled='true' and aria-describedby pointing to a tooltip explaining why.
Progress bar	ExportProgress uses role='progressbar' aria-valuenow={progress} aria-valuemin='0' aria-valuemax='100'.
Toast notifications	Rendered in an aria-live='polite' region so screen readers announce them without interrupting.
Colour contrast	Purple #534AB7 on white: contrast ratio 5.2:1. All text meets 4.5:1

Requirement	Implementation
	minimum.

12. Implementation Phases

Phase 1

Remove Coming Soon & Wire Export Button

Days 1–2

- grep all Coming Soon instances and create a single PR removing every placeholder
- Remove disabled states from download buttons in FileActionMenu and FileDetailHeader
- Create ExportButton component (stub — opens console.log on click for now)
- Place ExportButton in the file table row action menu
- Place ExportButton in the file detail header action bar
- Create export.types.ts with ExportFormat, ExportState, FileType types
- Create formats.ts with EXPORT_FORMATS constant and isFormatAvailable utility
- Create exportApi.ts with initiateExport and getStatus (can point to a mock server)
- PR review + merge — this is shippable on its own

Phase 2

Format Selector & Modal Shell

Days 2–3

- Install Radix UI Dialog (or confirm Headless UI already in project)
- Build FormatSelector with grouped vector/raster sections
- Apply active/disabled/selected visual states per design rules in section 5.3
- Build ExportModal shell — wires open/close state, renders FormatSelector in body
- Connect ExportButton to open ExportModal
- Wire Export button inside modal to call handleExport (stub — just logs for now)
- Wire Cancel button to close modal
- Test keyboard navigation: Tab between format options, ESC closes, Enter selects
- Test screen reader: VoiceOver/NVDA announces modal title and format options correctly

Phase 3

Polling Hook & Progress UI

Days 3–4

- Install @tanstack/react-query if not already in project
- Build useExportStatus hook with refetchInterval logic (section 7.1)
- Build useExport hook — calls initiateExport, stores taskId, activates polling
- Connect handleExport in ExportModal to useExport hook

- Build ExportProgress component — spinner for QUEUED, progress bar for CONVERTING
- Wire ExportModal body to swap FormatSelector for ExportProgress when state transitions
- Implement the 10-minute timeout in useExportStatus (section 7.2)
- Test with mocked API responses: simulate QUEUED → CONVERTING (25%) → CONVERTING (80%) → SUCCESS
- Test timeout: mock API that never resolves — verify timeout fires after 10 minutes

Phase 4

Notifications & Background Polling

Days 5–6

- Build ExportSuccessToast with persistent display and Download CTA button
- Build ExportErrorToast with persistent display and Try Again button
- Wire success toast to fire when polling hook returns status = success
- Wire error toast to fire when polling hook returns status = failure or timeout
- Build global ExportStore (Zustand) to track in-flight tasks across navigation
- Implement close-while-converting confirmation dialog (section 6.2)
- Verify background polling: open modal, start export, close modal, navigate to different page — toast fires when complete
- Test error recovery: click Try Again in error toast — modal re-opens at SELECTING state

Phase 5

Polish, Accessibility & QA

Day 7

- Run axe-core accessibility audit on ExportModal — fix all WCAG AA violations
- Test full flow with VoiceOver (macOS) and NVDA (Windows)
- Test on mobile viewport (375px) — modal must be full-width with correct padding
- Test rapid click: clicking Export As multiple times should not create duplicate tasks
- Test concurrent exports: two files, two modals — each tracks independently
- Test expired download link: simulate clicking a link after 24h — verify friendly message
- Cross-browser smoke test: Chrome, Firefox, Safari, Edge
- Remove all console.log stubs left from phases 1–2
- Code review and merge

13. Acceptance Criteria

13.1 Functional

- No 'Coming Soon' text appears anywhere in the file management UI
- Export As button is visible and clickable on every file row and in the file detail header

- Modal opens with correct format options pre-filtered by file type — vector files never see raster formats as enabled options
- Clicking Export closes the format selector and shows a spinner immediately (no delay)
- Progress bar advances in real-time as the API returns progress updates
- Closing the modal while conversion is in progress triggers a confirmation prompt
- Success toast appears with a working download link after conversion completes
- Clicking Download in the success toast opens the file in a new tab
- Error toast appears with a readable error message and a functional Try Again button
- Background polling continues after modal is closed — toast fires regardless of current page

13.2 Non-functional

Criterion	Threshold
Modal open animation	< 150ms from click to modal visible
Export button → spinner	< 200ms from click to spinner showing (optimistic UI)
Polling interval	Exactly 3 seconds between status checks — verify in Network tab
Timeout	Polling stops and error state shown after exactly 10 minutes of no terminal state
Accessibility	0 WCAG AA violations in axe-core audit
Bundle size impact	New feature code adds < 15KB gzipped to the main bundle
Mobile layout	Modal renders correctly at 375px width — no horizontal scroll

14. Development Mocking Strategy

The frontend work can proceed in parallel with the backend. Use MSW (Mock Service Worker) to intercept API calls during development:

```
// src/mocks/handlers/export.ts
import { http, HttpResponse, delay } from 'msw'

export const exportHandlers = [
  http.post('/api/export', async () => {
    await delay(300)
    return HttpResponse.json({ task_id: 'mock-task-123', status: 'queued' },
    { status: 202 })
  }),

  http.get('/api/export/status/:taskId', async ({ params }) => {
    await delay(1000)
    // Cycle through states on each call for realistic simulation
    return HttpResponse.json({ status: 'progress', progress: 45, message:
    'Converting...' })
  })
]
```

```
    }},  
  ]
```

Mock scenario	How to trigger
Happy path — fast conversion	status: progress 20 → 80 → success in 3 polling cycles
Slow conversion	Return status: progress for 15+ cycles before success
Conversion failure	Return status: failure with error message after 2 cycles
Network error	Use MSW network error handler to simulate fetch failure
Timeout	Always return status: progress — timeout fires after 10 minutes

15. Risks & Mitigations

Risk	Likelihood / Impact / Mitigation
Backend not ready when frontend starts	High / Low — Use MSW mocks from day 1. Feature is fully testable without a live backend.
Polling interval too aggressive on mobile	Low / Medium — 3s is fine on desktop. On slow connections React Query auto-pauses when tab is hidden (visibilitychange).
User closes tab mid-conversion	Medium / Low — Tab closure kills the poll. The signed URL is still generated. User must re-initiate export. Accept this limitation for v1.
Large number of in-flight exports filling toast stack	Low / Medium — Limit global ExportStore to 5 active tasks. Show a queue badge on the Export button if limit is reached.
Signed URL shown in toast is copied and shared	Low / Medium — This is intentional. 24h TTL limits exposure. No additional mitigation needed in v1.
Radix UI / Headless UI version conflict with existing deps	Low / High — Check peer deps before installing. If conflict, build a plain HTML dialog with manual focus management.

End of document — GriidAi Frontend Team